Johnson&Johnson
MedTech

Python Cheat Sheet for Diagnostics and Troubleshooting

Use this Python Cheat Sheet as a quick reference while working on diagnostics and optimizations for robotic systems. These tools and techniques will support you in identifying performance issues, testing solutions, and documenting your progress. Practice and experimentation are key—don't hesitate to try different approaches as you enhance your diagnostics skills!

Basic Python Syntax

Comments

Use # to add comments, making code easier to read.

```python
# This is a comment explaining the code
```

Variables

Assign values to variables to store data.

```python
response_time = 0.12  # Stores a response time value
```

Control Structures

If Statements

Use *if, elif,* and *else* for conditional logic.

```python
if response_time > 0.15:
    print("Response time is too high!")
```

For Loops

Repeat Actions for items in a list.

```python
commands = ["move_arm", "rotate_joint", "adjust_grip"]
for command in commands:
    print(command)
```

While Loops

Run code as long as a condition is true.

```python
while response_time > 0.15:
    print("Adjusting response time...")
    response_time -= 0.01
```

Functions

Defining Functions

Functions organize and reuse code.

```python
def check_response_time(command):
    start_time = time.time()
    # Simulated execution
    response_time = time.time() - start_time
    return response_time
```

Calling Functions

Run a function by using its name.

```python
check_response_time("move_arm")
```

Working with Time

Measure Time

Use *time.time() to measure elapsed time.*

```python
start_time = time.time()
# Execute code here
elapsed_time = time.time() - start_time
print("Elapsed time:", elapsed_time)
```

Lists and Basic Operations

Creating Lists

Store multiple items in a list.

```python
commands = ["move_arm", "rotate_joint", "adjust_grip"]
```

Accessing List Items

Access items by their index.

```python
first_command = commands[0]
```

Looping Through Lists

Iterate over each item in a list.

```python
for command in commands:
    print(command)
```

Error Handling

Try and Except

Use *try* and *except* to handle errors.

```python
try:
    response_time = check_response_time("move_arm")
except Exception as e:
    print("An error occurred:", e)
```

Diagnostic Tools

Root Cause Analysis

Focus on areas likely to cause delays.

```python
if response_time > 0.15:
    print("Investigate command: rotate_joint")
```

Iterative Testing

Break problems into manageable parts and test segment.

```python
commands = ["move_arm", "rotate_joint", "adjust_grip"]
for cmd in commands:
    print(f"{cmd}: {check_response_time(cmd)} seconds")
```

Simulation Tools

Logging Results

Track and log response times for analysis.

```python
log = {}
commands = ["move_arm", "rotate_joint", "adjust_grip"]
for cmd in commands:
    log[cmd] = check_response_time(cmd)
print("Response Time Log:", log)
```

Optimization Testing

Test modified code to evaluate improvements.

```python
def optimized_command(command):
    print(f"Optimizing {command}...")
    return check_response_time(command)
```

Quick Reference Table

Command	Description	Example
time.time()	Measure elapsed time.	start_time = time.time()
try...except	Handle errors during execution.	See Error Handling example.
for loops	Iterate over lists or sequences.	for cmd in commands:
if...elif...else	Implement conditional logic.	See If Statements example.

Best Practices

1. Use descriptive variable names for clarity.
2. Break problems into smaller steps using functions.
3. Regular test and document your progress.
4. Keep your code organized and readable.